RAPPORT TECHNIQUE D'ÉCOSYSTEME

1. PRESENTATION GENERALE DU PROJET

Le projet DigiMarket s'inscrit dans une phase de transformation numérique visant à diversifier les activités de l'entreprise, historiquement spécialisée dans le matériel informatique (PC hardware), vers une offre e-commerce complète. Le pivot de cette stratégie est le développement d'une API REST performante, centralisant la gestion du catalogue, des flux clients et du cycle de vie des commandes.L'infrastructure logicielle repose sur un socle technologique moderne et éprouvé :

- **Langage :** Python
- **Framework Backend :** Flask (Architecture modulaire par Blueprints)
- **Gestion de Base de Données :** SQLite
- **ORM (Object-Relational Mapping) :** SQLAlchemy
- **Authentification et Sécurité :** PyJWT (JSON Web Tokens) et Werkzeug
- **Génération de Données :** Faker (pour la population des environnements de test)

2. ARCHITECTURE LOGICIELLE : MODELE MVC MODULAIRE

L'application adopte un pattern **Modèle-Vue-Contrôleur (MVC)** modulaire, garantissant une séparation stricte des préoccupations (SOC) et une maintenabilité accrue du code source.

Segmentation via les Blueprints

L'organisation logique est assurée par l'utilisation de **Blueprints** Flask, permettant d'isoler les domaines métier :

- **auth_bp :** Gestion des identités, inscription et flux d'authentification.
- **products_bp :** Administration du catalogue produit et consultation.
- **orders_bp :** Logique transactionnelle, gestion du panier et validation des stocks.
- **web_bp :** Routes dédiées au service des templates HTML pour l'interface frontend.

MODELISATION DES DONNEES ET INTEGRITE REFERENTIELLE

Le schéma relationnel est structuré autour de quatre entités clés au sein de SQLAlchemy, utilisant des clés étrangères (Foreign Keys) pour assurer la cohérence des données :

- **Utilisateur :** Entité pivot incluant l'email (unique), le mot de passe haché et le rôle (RBAC).
- **Produit :** Stocke les spécificités techniques, le prix unitaire et l'état des stocks.
- **Commande :** Enregistre l'en-tête transactionnel, lié à un **Utilisateur** via une relation One-to-Many (utilisateur_id).
- **LigneCommande :** Détaille le contenu des commandes, liant les entités **Commande** et **Produit** pour assurer la traçabilité des prix et quantités au moment de l'achat.

Structure des Fichiers de l'Architecture

Dossier / Fichier	Rôle MVC	Description
app/database/model.py	**Modèle**	Définition des classes SQLAlchemy et contraintes d'intégrité.
app/products/routes.py	**Contrôleur**	Logique métier de gestion du catalogue.
app/orders/routes.py	**Contrôleur**	Orchestration des commandes et validation des stocks.
app/auth/routes.py	**Contrôleur**	Gestion des accès et sécurité.
app/templates/	**Vue**	Rendu des interfaces web via Jinja2.
app/app_utils.py	**Utilities**	Services transverses (JWT, helpers de recherche, filtrage).

3. SECURITE LOGICIELLE ET AUTHENTIFICATION

La sécurisation de l'écosystème DigiMarket est une priorité architecturale, articulée autour de protocoles de sécurité standards.

Authentification JWT (JSON Web Tokens)

Le système implémente une authentification sans état via l'algorithme **HS256** . Le processus repose sur une SECRET_PHRASE serveur permettant de signer les jetons. Lors de chaque requête protégée, le client doit transmettre le jeton dans l'en-tête Authorization. Le serveur décode alors le payload pour identifier l'utilisateur et vérifier la date d'expiration.

Contrôle d'Accès Basé sur les Rôles (RBAC)

La gestion des droits est pilotée par le décorateur @requires_authorization. Ce mécanisme ne se contente pas de vérifier la validité du token ; il est responsable de l'injection des données de session (data_in_token) directement dans les fonctions de route. Cela permet de restreindre l'accès aux opérations sensibles (création/suppression de produits, mise à jour des statuts de commande) aux seuls profils possédant le rôle "Admin".

Protection Contre les Vulnérabilités

- **Injection SQL :** L'utilisation systématique de l'ORM SQLAlchemy neutralise les risques d'injection en utilisant des requêtes paramétrées.
- **Hachage des Mots de Passe :** Aucun mot de passe n'est stocké en clair grâce à l'implémentation des algorithmes de hachage de la bibliothèque Werkzeug.

---

4. SPECIFICATIONS DE L'API REST

4.1 Authentification

Points de terminaison pour la gestion des utilisateurs et des sessions.

- **Inscription :** POST /api/auth/register
- **Connexion :** POST /api/auth/login**Exemple de réponse (Register) :**


```
[
  {
    "date_creation": "Wed, 22 Apr 2026 19:51:12 GMT",
    "email": "tested_client_2@test.net",
    "id": 13,
    "nom": "Tested Client 2",
    "role": "client"
  }
]
```


4.2 Catalogue Produits

La consultation est ouverte au public, tandis que les opérations d'écriture sont restreintes aux administrateurs.**Détails d'un produit spécifique (GET /api/produits/2) :**


```
[
  {
    "categorie": "Stockage",
    "date_creation": "Tue, 07 Apr 2026 17:31:38 GMT",
    "description": "High-performance Stockage hardware. Ideal for gaming and professional video editing. Manufacturer warranty included. Specs: coin 59 Plus.",
    "id": 2,
    "nom": "Stockage Corsair - SSD NVMe 1To",
    "prix": 147.6,
    "quantite_stock": 3
  }
]
```

4.3 Gestion des Commandes

Le processus de commande inclut une vérification critique des stocks en temps réel.**Exemple de création de commande réussie (POST /api/commandes) :**

```
[
  {
    "adresse_livraison": "123, rue de la Paix, Paris",
    "date_commande": "Wed, 22 Apr 2026 19:51:12 GMT",
    "id": 11,
    "statut": "en_attente",
    "utilisateur_id": 11
  }
]
```

Exemple d'erreur de stock (403/400) :

```
{
  "error": "200 > stock of 30 for product id 56."
}
```

---

5. GUIDE D'INSTALLATION COMPLET

Déploiement de l'environnement de développement local :**Étape 1 : Clonage du dépôt Git**

```
git clone https://github.com/JeanRosselVallee/BlentPythonProject.git
```

Étape 2 : Configuration de l'environnement virtuel

```
python -m venv .venv
# Sur Windows :
.venv\Scripts\activate
# Sur Unix/macOS :
source .venv/bin/activate
```

Étape 3 : Installation des dépendances

```
pip install -r requirements.txt
```

Étape 4 : Initialisation et population de la base de données

```
python -m app.database.populate
```

Étape 5 : Exécution du serveur

```
python run.py
```

6. TESTS ET VALIDATION

La robustesse de l'API est validée par une suite de tests automatisés via pytest. Les comportements de sécurité et la logique métier sont consignés dans pytest.log.**Synthèse de la stratégie de test :**

- **Intégrité RBAC :** Validation du blocage des requêtes non autorisées. Conformément aux logs, une tentative de création de produit par un client retourne systématiquement une erreur 403 : { "error": "User {email} not authorized to create products." }.
- **Validation des flux :** Test complet du cycle de commande, de l'authentification à la création de lignes de commande.
- **Gestion des stocks :** Vérification des limites de quantités et refus des transactions excédant les disponibilités réelles.
- **Isolation :** Confirmation que les clients ne peuvent pas accéder aux données de commande d'autrui, respectant ainsi la confidentialité des données.L'intégralité des tests critiques présente un statut de réussite, confirmant la viabilité de l'écosystème pour une mise en production.